

Project DSAI

Matching User Preferences: Optimal Assignment of Students to Projects

Author:

PEREIRA Gabriel, DELEAGE Gwendal, MEJRI Jasser, SESOKO Rafael, RIVIERE
Sébastien

Class: Data Science and Artificial Intelligence

School Year: 2025/26

Date: 23.06.2026



School: Télécom Paris

Teachers: Ms. Ekhine IRUROSZKI ARRIETA,

Contents

1	Introduction, Hypotheses, and Motivation	4
1.1	Problem context	4
1.2	Project Question	4
1.3	Concrete applications	4
1.4	Motivation	5
1.5	Scope of the final report	5
2	Phase 1: Implementation Process	5
2.1	Core abstractions and the matching pipeline	5
2.2	Project organisation and repository structure	6
2.3	Data	6
2.3.1	Real data	6
2.3.2	Synthetic data	7
2.4	Classical mechanisms	7
2.4.1	Serial Dictatorship	7
2.4.2	Gale-Shapley deferred acceptance	8
2.4.3	Top Trading Cycles	8
2.5	Evaluation metrics	9
3	Phase 2 : Extensions and Analysis	10
3.1	Extension 1: Two-sided preferences with priorities (hr)	10
3.2	Extension 2: Partial preferences and ties (hrt)	10
3.3	Extension 3: Vetoes and constraints	10
3.4	Extension 4: Coalitions and groups	11
3.5	Extension 5: Statistical / Noisy-Preference Setting	11
4	Experimental Setup and Results	11
4.1	Experimental protocol	11
4.2	Results on real data	11
4.3	Results on synthetic data	12
4.3.1	Effect of preference noise	12
4.3.2	Effect of project capacity	13
4.3.3	Runtime	13
4.4	Interpretation	13
5	Challenges and findings	14
5.1	Technical challenges	14
5.2	Methodological challenges	14
5.3	Main findings	14
5.4	Limitations	14
6	Alignment with the initial project plan	15
6.1	What was initially planned	15
6.2	What was completed	15
6.3	What was not completed and why	15
6.4	What we would do differently	15
6.5	Lessons learned	16
7	Recommendation for the Télécom Paris Use Case	16

8 Conclusion 16

9 Team Contributions 16

10 Use of AI Tools 18

10.1 Use of AI tools in the code 18

10.2 Use of AI tools in the report 18

10.3 Transparency statement 18

List of References 18

1 Introduction, Hypotheses, and Motivation

1.1 Problem context

The motivation of this project is the assignment of students to projects at Télécom Paris. Each student submits an ordered preference list over the available projects. We also suppose here that the projects have preferences over the students so the goal is to find a stable matching considering those two constraints.

A matching is, in this case, a bijection from the students to the projects. The matching is **not stable** if :

1. there is a student X who prefers the given project of a student Y over his project and
2. the student Y also prefers the project given to X over his assigned project.

In other words, a matching is **stable** when there does not exist any pair (X, Y) which both prefer each other project to their current assigned project under the matching.

1.2 Project Question

The question to be answered here is : how do we choose the best classical mechanisms in order to decide the assignments of students to projects ? We will compare different algorithms such as Serial Dictatorship, Gale-Shapley and Top Trading Cycles. Moreover we will manage to implement other constraints such as partial preferences, vetoes or coalitions. Finally we can also ask us about the guarantees offered by those algorithms in order to find the best suited one for practical use.

Another question is the goal of this matching : do we want to maximize the student satisfaction ? projects satisfaction ? both satisfactions ? or do we want to minimize the unsatisfaction ? Depending on the metric we will choose, the most appropriate algorithm will change.

1.3 Concrete applications

The stable matching problem is present in many cases, the most obvious ones are Parcoursup (the french platform to assign high school students to colleges / universities) or the CROUS affectation in which we enumerate wishes and then the institutions choose who they will accept first, following given criteria.

Parcoursup is a typical example, each high school student order his wishes (Universities, Preparatory Classes, BTS (Advanced technician's certificate), ...) and each educational establishment order the candidates following different academic criteria. They all have their local algorithm whether they prefer students that live close to the site, or if they prefer good marks in physics instead of history, ... There are waiting lists because a student can accept temporarily an offer, waiting to get accepted in a better wish, and so the other unaccepted students are placed in waiting lists. Moreover the establishments have to follow certain constraint such as the number of scholarship students, a geographic quota, etc...

The CROUS affectation is a bit different, students are ranked following social criteria (scholarship students, disabled, distance between home - school, ...). Students can rank a residence following its location or its type (one-roomed studio, residence with common kitchen, collocation, etc.).

1.4 Motivation

Serial Dictatorship (SD), Gale-Shapley (GS) and Top Trading Cycles (TTC) are algorithms that don't aim to the same results. Where SD is Pareto-optimal, it is not necessarily stable, GS guarantees stability (no blocking pairs) at the cost of this Pareto-optimality. Finally, TTC restores Pareto-optimality from a stable matching at the cost of reintroducing blocking pairs.

- Pareto-optimality appears when the satisfaction of one side of the matching is optimized : we don't have a cycle of people that would be happier if they could exchange their assignments between each other. This is different from blocking pairs in the fact that Pareto-optimality is one-sided and not bounded to a length of two.

Our main theoretical groundwork is the Chapter 14 of the Handbook of computational social choice¹, which classifies bipartite matching problems with preferences into one-sided (House Allocation, ha) and two-sided (Hospitals/Residents, hr) variants and presents the canonical algorithms.

1.5 Scope of the final report

This report covers the work completed across three phases.

- Phase 1 (classical mechanisms, weeks 1–5) consisted in implementing Serial Dictatorship, Gale–Shapley (with per-project school priorities), and Top Trading Cycles. We then tested them, and evaluated on both real and synthetic data. A lot of different useful metrics have also been implemented in order to compare those algorithms.
- Phase 2 (extensions and analysis, weeks 6–8) consisted in implementing extensions to the initial problem such as partial preferences, vetoes and coalitions. There have also been some reflexions about a statistical algorithm of machine learning and deep learning but the lack of time and the difficulty to find a metric for convergence have break us.
- Phase 3 (synthesis, week 9) produced this report and the accompanying oral presentation, with a final recommendation for the Télécom Paris use case.

2 Phase 1: Implementation Process

2.1 Core abstractions and the matching pipeline

Led by Gabriel Viterbo, the implementation is organised around two core data structures and three modules that connect them. The Instance class encodes the input of a matching problem (students, projects with their capacities, student preferences) as an immutable object that validates its own consistency at construction time. Here, the immutability plays a big role into maintaining the code well modularized.

The Matching class represents the output: a mutable assignment that mechanisms build incrementally through assign and unassign primitives, and that exposes higher-level queries used by both algorithms and metrics. The three modules then form a linear pipeline: loaders turn raw CSV data (real or synthetic) into an Instance; each mechanism in mechanisms is a function from Instance to Matching implementing one allocation logic; and metrics consumes a Matching to produce scalar evaluations. This uniform

¹[1] U. E. J. L. A. D. P. Felix Brandt Vincent Conitzer, *Handbook of computational social choice*. Cambridge University Press, 2016.

Instance → Matching contract is what makes the three classical mechanisms directly comparable on a common footing, and what allowed us to develop the stages of the pipeline in parallel.

2.2 Project organisation and repository structure

The project is implemented mainly in Python 3.10, and we opted for a private repository (gviterbo/matching_project on GitHub) and it is organised as follows:

/src/core/	Core module containing definitions of immutable classes (Student and Project) and the mutable Matching class.
/src/mechanisms/	Implementation of the classical matching algorithms: Serial Dictatorship, Gale-Shapley, and adapted Top Trading Cycles.
/src/metrics/	Evaluation module comprising seven cost functions implemented in cost.py, three stability analysis functions in stability.py, and empirical parameter estimation for the Mallows model in inference.py.
/src/data/	Module responsible for dataset loading and synthetic data generation through the loaders.py file.
/src/top_k_mallows/	Implementation of utilities for the Mallows model, including synthetic preference generators (mallows_kendall.py and mallows_hamming.py).
/notebooks/	Interactive documentation containing notebooks for exploratory data analysis (Data_analysis.ipynb) and Phase 1 experimental report (report_phase_1.ipynb).
/tests/	Suite of unit tests using pytest to ensure the correctness of implementations.
/data/	Data repository containing the anonymized DSAI dataset (projects-dsai-2026-anon.csv) used in experiments.

Table: Repository main structure

2.3 Data

2.3.1 Real data

The real dataset consists of anonymised preference data from the DSAI project matching at Télécom Paris (April 2026). It is stored as a CSV file with three columns: stu_id (anonymised student identifier), proj_id (project identifier), and points (an integer between 0 and 100 representing the student's preference weight for that project, where 100 is the highest preference).

PRJ_002	100	STU_074
PRJ_003	85	STU_074
PRJ_012	80	STU_074
⋮	⋮	⋮

Table: Dataset structure example

No GPA data was available for the real dataset. Therefore, school-side priorities were not activated in the real-data experiments or generated according to a random order of students.

2.3.2 Synthetic data

Synthetic instances are generated using the Mallows model with Kendall distance (Mallows1957). The dispersion parameter $\phi \in (0, 1)$ controls the degree of consensus among student preferences: $\phi \rightarrow 0$ produces near-identical preference lists (high consensus), while $\phi \rightarrow 1$ produces approximately uniform random preferences. The consensus ranking σ_0 defaults to the identity permutation. Student GPA values are drawn uniformly at random from $\{0.00, 0.01, \dots, 4.00\}$ (without replacement), and a single global GPA-based priority list (highest GPA first) is shared across all projects, forming the school_priorities dictionary. This allows the GS and TTC mechanisms to exploit priority information.

2.4 Classical mechanisms

2.4.1 Serial Dictatorship

Serial Dictatorship (serial_dictatorship.py) processes students in a fixed order and assigns each student their most preferred project that still has available capacity, decrementing the project's remaining capacity counter.

Input: an instance I with students S , projects P with capacities $c : P \rightarrow \mathbb{N}$, student preferences $\text{pref} : S \rightarrow P^*$; an ordering π of the students.

Output: a matching $M : S \rightarrow P \cup \{\perp\}$.

Serial_Dictatorship(I, π):

1. $M \leftarrow \emptyset$
 2. $\text{cap}[p] \leftarrow c(p)$ for each $p \in P$
 3. **for** each student s in the order π **do**
 4. **for** each project p in $\text{pref}(s)$ **do**
 5. **if** $\text{cap}[p] > 0$ **then**
 6. $M(s) \leftarrow p$
 7. $\text{cap}[p] \leftarrow \text{cap}[p] - 1$
 8. **break**
 9. **if** s was not assigned **then** $M(s) \leftarrow \perp$
 10. **return** M
-

Algorithm 1: Serial Dictatorship

The ordering policy is a parameter: if no order is provided, the default is the input order of students in the Instance. Alternative orderings (e.g. by GPA, or a random permutation) can be passed explicitly. The algorithm runs in $O(n \cdot m)$ time in the worst case, where n is the number of students and m the number of projects.

2.4.2 Gale-Shapley deferred acceptance

The Gale–Shapley implementation (`gale_shapley.py`) follows the applicant-proposing variant. Free students are kept in a queue and propose to the next project that has not rejected them; if the project is full, the proposer displaces its lowest-priority holder when preferred, and the displaced student returns to the queue.

Input: an instance I with students S , projects P with capacities $c : P \rightarrow \mathbb{N}$, student preferences $\text{pref} : S \rightarrow P^*$, and project priorities $\text{prio} : P \rightarrow S^*$.
Output: a stable matching $M : S \rightarrow P \cup \{\perp\}$.

Gale_Shapley(I):

1. $M \leftarrow \emptyset$
 2. $\text{cap}[p] \leftarrow c(p)$ for each $p \in P$
 3. $F \leftarrow S$
 4. **while** $F \neq \emptyset$ **do**
 5. pick a student s from F with a non-exhausted preference list
 6. **if** no such s exists **then break**
 7. $p \leftarrow$ next project in $\text{pref}(s)$ to which s has not yet proposed
 8. **if** $\text{cap}[p] > 0$ **then**
 9. $M(s) \leftarrow p$; $\text{cap}[p] \leftarrow \text{cap}[p] - 1$; remove s from F
 10. **else**
 11. $s' \leftarrow \arg \max_{s'' \in M^{-1}(p)} \text{rank of } s'' \text{ in } \text{prio}(p)$
 12. **if** s is ranked above s' in $\text{prio}(p)$ **then**
 13. $M(s') \leftarrow \perp$; $M(s) \leftarrow p$
 14. remove s from F ; add s' to F
 15. **for each** $s \in S$ with $M(s)$ undefined **do** $M(s) \leftarrow \perp$
 16. **return** M
-

Algorithm 2: Gale–Shapley (applicant-proposing)

The algorithm terminates when the queue empties, in $O(nm)$ proposals. Priorities are read from `school_priorities`, or derived from student GPA via the `useGPA` flag (a single global ranking shared by all projects). Without priorities, the run degenerates to a first-come-first-served allocation along the queue, used only as a baseline against Serial Dictatorship.

2.4.3 Top Trading Cycles

The TTC implementation (`ttc_from_matching`) is looking to upgrade its input matching, seeking for cycles of assignment exchanges in order to improve the satisfaction of the students. We first operate a pre-processing step to get a first matching (typically the GS output), seeking Pareto-improving exchanges. Each matched student points to their most preferred project among those ranked strictly above their

current assignment; each project points to the highest-priority student currently assigned to it who still wishes to improve. Cycles in this graph are detected by depth-first search with path tracking. Depth-first search explores the exchange graph by following arcs as deep as possible, maintaining a running path of visited nodes. At each step, if the next neighbor is the starting node and the path has at least 2 nodes, a cycle is closed and recorded. Otherwise, the algorithm backtracks and tries other neighbors, exhaustively enumerating all simple cycles. When a cycle is found, the students in the cycle swap to their pointed project; their preference truncation is updated and the search resumes. Students who no longer have a strictly better option are removed from the search. The algorithm terminates when no student can improve.

Input: an instance I with students S , projects P , student preferences $\text{pref} : S \rightarrow P^*$, project priorities $\text{prio} : P \rightarrow S^*$, and an initial matching $M_0 : S \rightarrow P$.

Output: a Pareto-optimal matching $M : S \rightarrow P$ that weakly improves M_0 for every student.

TTC(I, M_0):

1. $M \leftarrow M_0$
 2. **repeat**
 3. build the exchange graph G : for each $s \in S$, add arc $s \rightarrow t$ iff s strictly prefers $M(t)$ to $M(s)$
 4. $\mathcal{C} \leftarrow \text{FindCycles}(G)$
 5. **if** $\mathcal{C} = \emptyset$ **then break**
 6. **for each** cycle $(s_1, s_2, \dots, s_k) \in \mathcal{C}$ **do**
 7. **for** $i \leftarrow 1$ **to** k **do** $M(s_i) \leftarrow M(s_{i+1 \bmod k})$
 8. **return** M
-

FindCycles(G):

1. $\mathcal{C} \leftarrow \emptyset$; visited $\leftarrow \emptyset$
 2. **for each** $s \in S$ with $s \notin \text{visited}$ **do**
 3. run DFS from s with path tracking
 4. **if** a cycle $(s_1, \dots, s_k)$ is closed **then**
 5. $\mathcal{C} \leftarrow \mathcal{C} \cup \{(s_1, \dots, s_k)\}$
 6. mark all nodes in the DFS path as visited
 7. **return** $\mathcal{C}$
-

Algorithm 3: Top Trading Cycles (improvement from a given matching)

2.5 Evaluation metrics

The evaluation module in `src/metrics/` implements two complementary families of scores, all normalised to $[0, 1]$ with 0 optimal and 1 worst.

Cost and welfare (`cost.py`) quantifies the quality of an allocation from the students' side, in three groups:

- **Global efficiency:** the **rank-sum cost** $C_{\text{rank}} = \frac{1}{N(|P|-1)} \sum_{s=1}^N r_{s(p_s)}$ (with $r_{s(p)}$ the 0-indexed rank of p in s 's list and unmatched students taking rank $|P| - 1$); the **Borda welfare cost** (opportunity loss in Borda score); the **top- n miss rate** (fraction placed outside their top n).
- **Fairness:** the **max-rank cost** (worst rank suffered), the **squared rank-sum cost** (quadratic penalty on bad assignments), and the **Nash welfare cost** (geometric mean of satisfactions, favouring balance).
- **Feasibility:** the **unassigned rate**.

Stability (`stability.py`) requires explicit project priorities over students:

- **Blocking-pairs score:** fraction of (s, p) pairs where s prefers p to their assignment and p either has free capacity or prefers s to some current holder, normalised by $|S| \cdot |P|$.
- **Justified-envy score:** fraction of students who could displace a lower-priority holder of a more-preferred project, normalised by $|S|$.
- **Wasteful-capacity score:** fraction of empty slots desired by some unassigned or under-placed student, normalised by total capacity.

Without project priorities, stability metrics depend on arbitrary tie-breaking and are not reported.

3 Phase 2 : Extensions and Analysis

Note: This section reports the status of each Phase 2 extension. Table 1 gives an overview; sub-sections detail what was completed.

3.1 Extension 1: Two-sided preferences with priorities (hr)

The Gale–Shapley and Top Trading Cycles implementations both accept the `school_priorities` field of the Instance, turning the one-sided allocation into a genuine two-sided matching problem whenever per-project priority lists are provided. Synthetic experiments (Section 4.3) exercise this mode using GPA-derived priorities; the real DSAI dataset is run without priorities, since no GPA data was available.

3.2 Extension 2: Partial preferences and ties (hrt)

The partial preferences have been considered in the GS and TTC algorithm by considering that a project that is not ranked has a rank of +infinity (the student does not want this project). Because we don't look for an heuristic or for the precise rank in the SD, the unranked projects have been put in an arbitrary order at the end of the list of preferences.

3.3 Extension 3: Vetoes and constraints

This extension was not implemented due to time constraints after the midterm presentation. The architecture, however, makes the addition straightforward. Vetoes (a student refusing a specific project, or a project refusing a specific student) reduce to filtering the Instance before any mechanism is called: the vetoed project is removed from the student's preference list, or the vetoed student from the project's priority list. Since SD, GS and TTC all consume preferences and priorities as plain lists, no mechanism code needs to change. Forbidden pairs follow the same pattern.

3.4 Extension 4: Coalitions and groups

Led by Rafael Sesoko. This variant of the problem considers not only students' preferences, but also the fact that they may be divided into groups or coalitions. For this implementation, a core module was added to define a `CoalitionInstance`, which works similarly to a regular instance but also keeps a list of coalitions. A new variant of serial dictatorship was then added, using the groups defined in the coalition instance. To evaluate a group's preferences, we analyze each student in the group and rank the projects by summing their individual preference ranks.

3.5 Extension 5: Statistical / Noisy-Preference Setting

Led by Sébastien Rivière. This part evaluates the allocation mechanisms under varying noise conditions using the Mallows model to handle truncated synthetic preferences. Technically, an inference routine was developed to learn the dispersion parameter from the Télécom Paris DSAI dataset. The optimization yielded $\varphi = 0.88$, indicating highly diverse project interests and low consensus among students. This inferred value explains why outcomes align so closely on the real dataset: the natural diversity of student preferences at Télécom Paris drastically reduces structural allocation conflicts. To evaluate the robustness of the algorithms against varying environments, synthetic experiments varied $\varphi \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. The results show that Gale-Shapley (GS) remains universally robust, while Serial Dictatorship (SD) proves highly vulnerable to high-consensus settings ($\varphi = 0.1$), reaching a blocking-pairs score of 0.312.

4 Experimental Setup and Results

4.1 Experimental protocol

All experiments use Python 3.10 with pandas, numpy, and scipy. Mechanisms are run on the same instance to isolate algorithmic differences from instance-to-instance variability. Runtime measurements are averaged over 50 repetitions using `time.perf_counter`. Synthetic experiments use 10 independent random seeds per parameter configuration; reported values are means over these seeds. For the real dataset, the default student order (CSV input order) is used for SD; GS uses no school priorities (real GPA data is not available); TTC is initialised from the GS output. For synthetic data, GPA-based school priorities are activated (same global order for all projects), enabling meaningful stability measurements.

4.2 Results on real data

The real DSAI dataset (106 students, 21 projects, capacity 5 per project, total capacity 105) is nearly supply-constrained: exactly one student remains unmatched under all three mechanisms. Table 2 reports the metrics. All three mechanisms produce the same assignment on this instance. Table 2: Comparison of mechanisms on the real DSAI dataset (106 students, 21 projects, capacity= 5). All metrics normalised to $[0, 1]$; lower is better. Blocking-pairs and justified-envy scores are 0 by definition when no school priorities are available

RESULTS :

The three mechanisms yield identical outcomes on the real dataset, explained by two structural properties. First, no school-side priorities are available, so GS degenerates to a form of SD. Second, the abundant capacity ($c = 5$ across 21 projects for 106 students) creates enough slack that most students obtain a top-ranked project without competition, leaving little room for TTC to find improving cycles. The mean rank obtained is 3.7, with 43 of the 105 matched students receiving their first choice.

The absence of project rankings precludes the direct use of Gale–Shapley and TTC, so we explored two workarounds. The first constructs proxy project preferences from the scores students assign to projects, enabling standard GS. The second runs Serial Dictatorship repeatedly over randomly shuffled student orderings, evaluating each allocation by the share of students placed in their top three choices; a sufficient number of iterations reliably identifies low-cost allocations.

4.3 Results on synthetic data

To surface the algorithmic differences predicted by theory, we use synthetic instances where school priorities (GPA-based) are active, enabling GS to perform genuine priority-based displacement and TTC to seek Pareto improvements over the stable matching.

4.3.1 Effect of preference noise

φ	Rank-sum cost			Blocking-pairs score			Just. envy	
	SD	GS	TTC	SD	GS	TTC	SD	GS
0.1	0.480	0.484	0.484	0.312	0.000	0.000	0.810	0.000
0.3	0.416	0.430	0.430	0.271	0.000	0.000	0.763	0.000
0.5	0.320	0.324	0.324	0.199	0.000	0.000	0.650	0.000
0.7	0.190	0.189	0.189	0.112	0.000	0.000	0.453	0.000
0.9	0.086	0.093	0.093	0.049	0.000	0.000	0.237	0.000

Table 1: Synthetic experiments: effect of Mallows noise parameter φ (30 students, 10 projects, capacity 3). Means over 10 seeds. Lower is better for all metrics.

4.3.2 Effect of project capacity

c	φ	Rank-sum cost			Blocking-pairs score		
		SD	GS	TTC	SD	GS	TTC
1	0.3	0.708	0.713	0.713	0.305	0.000	0.000
1	0.5	0.658	0.668	0.668	0.279	0.000	0.000
1	0.7	0.596	0.619	0.619	0.251	0.000	0.000
2	0.3	0.422	0.422	0.422	0.236	0.000	0.000
2	0.5	0.335	0.317	0.317	0.175	0.000	0.000
2	0.7	0.194	0.195	0.195	0.105	0.000	0.000
5	0.3	0.109	0.107	0.107	0.082	0.000	0.000
5	0.5	0.057	0.056	0.056	0.038	0.000	0.000
5	0.7	0.015	0.018	0.018	0.012	0.000	0.000

Table 2: Means over 10 seeds for 20 students, 10 projects, $\varphi \in \{0.3, 0.5, 0.7\}$, varying capacity $c \in \{1, 2, 5\}$

At $c = 1$, half the students are left unmatched (50% unmatched rate); at $c \geq 2$, all students are matched.

4.3.3 Runtime

Instance	SD (ms)	GS (ms)	TTC (ms)
Real data (106 students, 21 projects)	0.03	0.15	5.64
Synthetic (100 students, 30 projects, $\varphi = 0.5$)	0.08	1.20	5.23

Table 3: Mean runtime per call (ms), averaged over 50 repetitions.

All three mechanisms run in well under 10ms on both instance sizes, making runtime a non-differentiating factor in the Télécom Paris context.

4.4 Interpretation

Synthetic experiments confirm the expected theoretical trade-offs: the Gale-Shapley (GS) algorithm consistently achieves zero blocking pairs and zero justified envy, validating its stability guarantee, whereas Serial Dictatorship (SD) fails stability by generating 0.05 to 0.31 blocking pairs depending on capacity and φ .

In terms of efficiency, SD and GS yield nearly identical rank-sum costs (differing by only 0.01 to 0.05), with SD occasionally leading when the input order happens to align with GPA priorities. Top Trading Cycles (TTC) fails to improve upon GS, producing identical outcomes and costs across all configurations, because all projects share identical student preferences based on uniform GPA priorities. Under these conditions, the structural prerequisites for Pareto-improving cycles—namely, misaligned project priorities across different students—are entirely absent, meaning GS already achieves a Pareto-optimal matching within TTC’s reachable set.

On the real dataset, all three mechanisms completely converge. This alignment is driven by the lack of school priorities and the diversity of students preferences.

5 Challenges and findings

5.1 Technical challenges

Several implementation challenges arose during Phase 1. In the TTC algorithm, the cycle- detection function initially iterated over the `school_priorities` dictionary without calling `.items()`, causing a `TypeError` on iteration. The Instance dataclass uses frozen fields, requiring `object.__setattr__` to initialise the internal lookup caches (`_student_by_id`, `_project_by_id`) in `__post_init__`. Gale-Shapley's use of a deque for the free-student queue required careful indexing to avoid accessing `instance.students` by integer index (students are stored as a tuple indexed by position, not by id). The `useGPA` branch in both GS and TTC was a source of subtle bugs when student GPA was `None`. The Mallows model implementation crashes for $\varphi = 0$ (division by zero in $\log(\varphi)$) and produces numerical instability near $\varphi = 1$; the safe range used in experiments is $\phi \in (0.05, 0.95)$.

5.2 Methodological challenges

A key methodological difficulty is that the real dataset lacks the structural conditions needed to differentiate the mechanisms: no school priorities, near-unit capacity, and partial preferences that are wide enough relative to the number of projects to avoid systematic conflicts. Constructing synthetic instances that reproduce the statistical properties of the real data while also enabling algorithmic differentiation required explicit parameter tuning (tight capacity, heterogeneous preferences, active school priorities). Comparing SD and GS fairly is non-trivial: the student ordering used by SD and the school priority ordering used by GS are not independent, so controlling one while varying the other is needed to isolate effects.

5.3 Main findings

1. GS eliminates blocking pairs; SD does not. On all synthetic instances with active school priorities, GS achieves a blocking-pairs score of 0.000 and a justified-envy score of 0.000, while SD exhibits scores of 0.05–0.31 and 0.24–0.81 respectively, depending on ϕ and capacity.
2. The welfare cost of stability is small. GS rank-sum costs differ from SD by at most 0.05 across tested configurations, confirming that the stability guarantee of GS does not come at a significant welfare price in these instances.
3. TTC provides no improvement over GS in these experiments. Under Mallows- model preferences with a shared GPA priority, GS already reaches a Pareto-optimal matching: TTC finds no improving cycles and produces identical outcomes to GS.
4. All three mechanisms are indistinguishable on the real dataset, due to the absence of school priorities and near-abundant capacity.

5.4 Limitations

The main limitations of this study are: (i) the real dataset is small (106 students, 21 projects) and lacks GPA or other priority data, preventing stability analysis on real data; (ii) synthetic experiments use a

single shared priority order across all projects, which reduces the likelihood of Pareto-improving cycles; (iii) TTC's capacity-aware cycle detection (allowing a project of capacity c to participate in up to c cycles per round) is not yet fully implemented; (iv) Phase 2 extensions were only partially completed, limiting the scope of the comparison; (v) no formal robustness analysis (confidence intervals, significance tests) was performed on the synthetic results.

6 Alignment with the initial project plan

6.1 What was initially planned

The project plan defined three phases.

- **Phase 1 (weeks 1–5, midterm deadline 29 May 2026)** covered the implementation and evaluation of Serial Dictatorship, Gale–Shapley, and Top Trading Cycles, together with evaluation metrics and experiments on both real and synthetic data. We've also gotten a head start on Phase 2 by implementing two-sided preferences with priorities directly.
- **Phase 2 (weeks 6–8)** extended the baseline with five directions: two-sided preferences with priorities, partial preferences and ties, vetoes and constraints, coalitions and groups, and the statistical/noisy-preference setting.
- **Phase 3 (week 9, deadline 26 June 2026)** produced the final report and oral presentation.

6.2 What was completed

- **Phase 1** was fully completed. SD, GS, and TTC are implemented, tested, and integrated into a reusable Python package; the evaluation module provides seven cost metrics and three stability scores; experiments on both real and synthetic data were conducted and reported.
- **Phase 2** was partly completed. The coalition part has been implemented for SD and the partial preferences has been implemented SD, TTC and GS.
- **Phase 3** the final report has been uploaded.

6.3 What was not completed and why

Several tasks from Phase 2 were not fully achieved due to time constraints, data limitations, and architectural complexities:

- **Extension 3 (Vetoes and constraints):** This part was started but left incomplete.
- **Extension 4 (Coalitions and groups):** Group logic was only implemented for the Serial Dictatorship mechanism, leaving GS and TTC without coalition support.
- **Machine Learning:** The deep learning approach was considered and then abandoned due to a lack of real data and sufficient time.
- **Statistical Evaluation:** Formal robustness analysis and confidence intervals were omitted.

6.4 What we would do differently

- **Earlier Scoping:** Evaluate Phase 2 extensions and alternative research directions (like machine learning) earlier in the timeline to better account for data constraints and tight schedules.
- **Data Preprocessing:** Analyze the real dataset at the very beginning of the project to detect the lack of priority and GPA information sooner.

- **Interface Definition:** Define clearer communication and integration interfaces between the mechanism execution modules and the evaluation metrics pipeline from the start.

6.5 Lessons learned

- **Technical:** Code modularity and the use of structured, immutable instances significantly simplify debugging and parallel teamwork.
- **Scientific:** Matching outcomes are highly dependent on the structural properties of the instance, such as preference diversity (φ) and capacity constraints (c).
- **Teamwork:** Iterative adjustments after milestone reviews and effective task allocation are essential to keep the project on track despite unfeasible pivots.

7 Recommendation for the Télécom Paris Use Case

Gale–Shapley and TTC are powerful methods for two-sided matching problems, and Shapley’s work on matching theory was recognized with the Nobel Prize in Economics. Mostly, at Télécom Paris, however, projects do not rank students: the allocation depends only on student preferences and the competition between them (in the order that they submitted the form). Stability, the central guarantee of GS, is no longer directly relevant, since no project-side preferences exist to form blocking pairs. The problem therefore reformulates as an optimization one, where the choice of cost function drives both complexity and method. Our approach reuses the tools we built: run Serial Dictatorship over many shuffled student orderings and keep the allocation minimizing the chosen cost. It offers no global-optimality guarantee, but it is simple, avoids heavier machinery, and works well in practice.

8 Conclusion

This project successfully delivered a robust simulation pipeline to analyze student-project allocation mechanisms. Through empirical evaluation and synthetic modeling using the Mallows distribution, we demonstrated that while Gale-Shapley guarantees stability in high-consensus environments, the natural diversity of preferences within the Télécom Paris DSAI cohort ($\varphi = 0.88$) naturally eliminates blocking pairs across all core algorithms. Despite the omission of certain Phase 2 extensions due to time and data constraints, the framework provides a solid foundation for understanding the operational trade-offs between efficiency, strategy-proofness, and stability in real-world matching markets.

9 Team Contributions

Ekhine Irurozki Arrieta — Project coordinator / supervisor

- Provided scientific guidance through our meetings, along with documentation resources and the Mallows model package. In the initial meeting, framed the core matching problem and suggested potential extensions. At the midterm review, assessed the implemented algorithms and helped guide the next steps under tight time constraints, notably suggesting a deeper empirical analysis of the real dataset to better understand the structure of actual preference lists.

Gabriel Pereira Viterbo

- **Phase 1:** The infrastructure of the code, which includes the git repository, the pipeline, the definition and implementation of Matching and Instance classes. Implementation of Serial Dictatorship and unit tests. Also in charge of the communication with the supervisor
- **Phase 2:** Extension to two-sided preferences with priorities (hr); comparison of student-optimal and school-optimal stable matching; coordination of the final integration; Chapter 2 of the final report.

Gwendal Deléage

- **Phase 1:** Implementation of TTC; unit tests and validation on small cases; presenting the TTC algorithm and the use cases of the algorithm during the midterm presentation. Implementing small cases to make compare SD, GS and TTC.
- **Phase 2:** Extension to partial preferences and ties (hrt); adding variants on TTC in order to implement the metrics of phase 1 in the search for better cycles. Writing some examples of comparison between GS and TTC. TTC now take as input the output matching of GS and tries to improve it rather than a given random matching. Coordination an taking part for the writing of the final report.

Jasser Mejri

- **Phase 1:** Definition and implementation of the cost functions and stability scores (src/metrics/); shared evaluation module; writing the evaluation methodology section of the midterm report; (Data_analysis.ipynb) midterm presentation (metrics).; adaptation of Phase 1 algorithms.
- **Phase 2:** I tried to work on a statistical learning model and extend our work to what we saw in the AI courses, but the task seemed overly difficult.

Rafael Sesoko

- **Phase 1:** The implementation of Gale-Shapley, including the student-proposing and project-proposing variants; handling capacities and priorities; unit tests.
- **Phase 2:** The extension to coalitions and groups; implementing coalition instances and serial dictatorship with coalitions; implementing small test cases to compare standard serial dictatorship with the coalition based version.

Sébastien Rivière

- **Phase 1:** Preparation and anonymization of the real DSAI project data; generation of synthetic data using the Mallows model (create_synthetic_instance); setup of experimental notebooks; writing the data and experimental results section of the midterm report; midterm presentation.
- **Phase 2:** Statistical / noisy-preference setting; conducted Mallows-model robustness experiments; investigated top- k truncation mechanics; implemented the empirical parameter inference routine to learn the true dispersion parameter ($\varphi = 0.88$) from the real DSAI dataset; analyzed mechanism performance and blocking-pairs convergence under varying preference noise thresholds.

Teamwork and organisation

The team collaborated using Git and GitHub (feature branches, pull requests), with coordination through shared documents and regular meetings with the supervisor. Task allocation followed the distribution in

the project plan and was refined iteratively after the midterm evaluation based on Phase 1 results. All members contributed to the writing of this report on Typst and to the preparation of the oral presentation.

10 Use of AI Tools

10.1 Use of AI tools in the code

During the development phase, we used GitHub Copilot and online Large Language Models (like Google Gemini). These tools were not used to generate our core algorithmic logic from scratch, but rather acted as an interactive support to help us refine and iterate on our own implementation, as well as:

- **Code Review & Debugging:** Helping to identify syntax errors and fix bugs.
- **Documentation:** Writing docstrings, adding comments to the code, and formatting explanations.
- **Conceptual Comprehension:** Clarifying some theoretical points about the matching mechanisms and the mathematical models behind them.

To make sure everything was correct, we verified every part of the code influenced by these tools. We performed manual code inspections and validated the correctness of the outputs using automated unit tests (pytest).

10.2 Use of AI tools in the report

For the report itself, online LLMs were only used for text editing and layout support:

- Helping to draft the initial outline of the document.
- Correcting English typos, fixing phrasing, and checking grammar.
- Helping with Typst syntax and formatting issues.

We reviewed and edited all text adjusted by the AI to ensure accuracy.

10.3 Transparency statement

As required by the course policy, we declare that AI tools were used only as technical and language support. All core analytical, mathematical, and empirical work was conducted by ourselves, and we remain responsible for the code and the report.

List of References

- [1] U. E. J. L. A. D. P. Felix Brandt Vincent Conitzer, *Handbook of computational social choice*. Cambridge University Press, 2016.